

Django Class-Based Views (CBV)

Django- **Class-Based View (CBV)** view function- **class** request handle

CBV- :

- Code reuse
- Inheritance
- Mixins reusable behavior
- CRUD application
- Authentication permission handling

1. Base / Generic Views

view CBV- base

1.1 View

View Django- basic CBV

```
javascript.snippet JAVASCRIPT

from django.views import View
class MyView(View):
    def get(self, request):
        # GET request handle
        pass
    def post(self, request):
        # POST request handle
        pass
```

Common HTTP methods:

```
javascript.snippet JAVASCRIPT

get()post()put()patch()delete()head()options()
```

View HTTP method method override

1.2 TemplateView

TemplateView template render

```
javascript.snippet JAVASCRIPT

from django.views.generic import TemplateView
class HomeView(TemplateView):
    template_name = "home.html"
```

URL:

```
javascript.snippet JAVASCRIPT

path("", HomeView.as_view(), name="home")
```

?

template logic

:

```
javascript.snippet JAVASCRIPT

Home Page About Page Contact Page
```

1.3 RedirectView

RedirectView URL URL- redirect

```
javascript.snippet JAVASCRIPT

from django.views.generic import RedirectView
class MyRedirectView(RedirectView):
    url = "/home/"
```

:

```
javascript.snippet JAVASCRIPT

class MyRedirectView(RedirectView):
    pattern_name = "home"
```

```
javascript.snippet JAVASCRIPT

RedirectView    Redirect
```

1.4 FormView

FormView Django form handle

```
javascript.snippet JAVASCRIPT

from django.views.generic import FormViewclass ContactView(FormView):    template_name
=
    "contact.html"    form_class = ContactForm    success_url = "/success/"
```

Form valid `form_valid()` method

```
javascript.snippet JAVASCRIPT

def form_valid(self, form):    # custom logic    return super().form_valid(form)
```

FormView **Model save**

Model save :

- CreateView
- UpdateView
- `form_valid()`- custom save logic

2. CRUD / Model-Based Generic Views

Django- generic views- **Create, Retrieve, Update, Delete (CRUD)**

2.1 ListView

object- list

```
javascript.snippet JAVASCRIPT

from django.views.generic import ListViewclass ProductListView(ListView):    model =
Product
    template_name = "products/product_list.html"    context_object_name = "products"
```

Default context :

```
javascript.snippet JAVASCRIPT
object_list
```

model :

```
javascript.snippet JAVASCRIPT
product_list
```

```
javascript.snippet JAVASCRIPT
Database Multiple Objects ListView Template
```

2.2 DetailView

object-

```
javascript.snippet JAVASCRIPT
from django.views.generic import DetailView
class ProductDetailView(DetailView):
    model = Product
    template_name = "products/product_detail.html"
```

URL- pk slug

```
javascript.snippet JAVASCRIPT
path("product/<int:pk>/", ProductDetailView.as_view(), name="product-detail")
```

```
javascript.snippet JAVASCRIPT
URL Object ID Database Single Object Template
```

2.3 CreateView

model object

```
javascript.snippet JAVASCRIPT

from django.views.generic import CreateViewclass ProductCreateView(CreateView):
model
    = Product    fields = ["name", "price"]    template_name = "products/product_form.html"
    success_url = "/products/"

CreateView:
```

```
javascript.snippet JAVASCRIPT

Form Display    User Input    Validation    Model Object Create    Redirect
```

2.4 UpdateView

Existing object update

```
javascript.snippet JAVASCRIPT

from django.views.generic import UpdateViewclass ProductUpdateView(UpdateView):
model
    = Product    fields = ["name", "price"]    template_name = "products/product_form.html"
    success_url = "/products/"
```

2.5 DeleteView

Existing object delete

```
javascript.snippet JAVASCRIPT

from django.views.generic import DeleteViewclass ProductDeleteView(DeleteView):
model
    = Product    template_name = "products/product_confirm_delete.html"    success_url = "/products/"
```

user- confirmation page

```
javascript.snippet JAVASCRIPT
Delete Request Confirmation POST Object Delete Redirect
```

3. Date-Based Generic Views

Django- date-based content- generic view

application- content- date/time

:

```
javascript.snippet JAVASCRIPT
BlogNewsEventsArticlesArchive
```

3.1 ArchiveIndexView

model- date-based object- archive/index

```
javascript.snippet JAVASCRIPT
from django.views.generic.dates import ArchiveIndexViewclass PostArchiveView(
ArchiveIndexView
):    model = Post    date_field = "published_at"
```

3.2 YearArchiveView

object

```
javascript.snippet JAVASCRIPT
from django.views.generic.dates import YearArchiveViewclass PostYearArchiveView(
YearArchiveView
):    model = Post    date_field = "published_at"
```

3.3 MonthArchiveView

object

```
from django.views.generic.dates import MonthArchiveViewclass PostMonthArchiveView(MonthArchiveView):    model = Post    date_field = "published_at"
```

3.4 WeekArchiveView

object

```
from django.views.generic.dates import WeekArchiveViewclass PostWeekArchiveView(WeekArchiveView):    model = Post    date_field = "published_at"
```

3.5 DayArchiveView

object

```
from django.views.generic.dates import DayArchiveViewclass PostDayArchiveView(DayArchiveView):    model = Post    date_field = "published_at"
```

3.6 TodayArchiveView

object

```
from django.views.generic.dates import TodayArchiveViewclass TodayPostView(TodayArchiveView):    model = Post    date_field = "published_at"
```

Date-Based Views Summary

```
javascript.snippet JAVASCRIPT

ArchiveIndexView Archive/ListYearArchiveView YearMonthArchiveView
MonthWeekArchiveView
WeekDayArchiveView DayTodayArchiveView Today
```

4. Authentication Generic Views

Django- authentication system- built-in generic views

module:

```
javascript.snippet JAVASCRIPT

django.contrib.auth.views
```

:

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.views import LoginView
```

4.1 LoginView

User login

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.views import LoginViewclass UserLoginView(LoginView):
    template_name
    = "accounts/login.html"
```

4.2 LogoutView

User logout

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.views import LogoutViewclass UserLogoutView(LogoutView):
    next_page
    = "/"
```

4.3 PasswordChangeView

Logged-in user password change

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.views import PasswordChangeViewclass MyPasswordChangeView(
    PasswordChangeView
):
    template_name = "accounts/password_change.html"
```

4.4 PasswordChangeDoneView

Password successfully change success page

4.5 PasswordResetView

Password reset process email/form handle

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.views import PasswordResetView
```

4.6 PasswordResetDoneView

Password reset email confirmation page

4.7 PasswordResetConfirmView

User email- reset link password

4.8 PasswordResetCompleteView

Password reset success page

Authentication Views Summary

javascript.snippet		JAVASCRIPT
LoginView	User LoginLogoutView	User LogoutPasswordChangeView
Change	Change	
PasswordPasswordChangeDoneView		
Password Change Success	PasswordResetView	Start Password
ResetPasswordResetDoneView		
Reset Email Sent	PasswordResetConfirmView	Set New
PasswordPasswordResetCompleteView		
Reset Completed		

5. Mixins -- Reusable Behaviors

Mixin reusable class CBV- combine behavior

Mixin standalone view

5.1 LoginRequiredMixin

User login view access

javascript.snippet		JAVASCRIPT
<pre>from django.contrib.auth.mixins import LoginRequiredMixin from django.views.generic import ListViewClass DashboardView(LoginRequiredMixin, ListView): model = Product</pre>		
:		

javascript.snippet		JAVASCRIPT
Not Logged In	Login Page	Logged In Dashboard

5.2 PermissionRequiredMixin

User- permission check

javascript.snippet

JAVASCRIPT

```
from django.contrib.auth.mixins import PermissionRequiredMixinclass ProductDeleteView(PermissionRequiredMixin, DeleteView):    model = Product    permission_required = "shop.delete_product"
```

5.3 UserPassesTestMixin

custom condition user access control

javascript.snippet

JAVASCRIPT

```
from django.contrib.auth.mixins import UserPassesTestMixinclass MyView(UserPassesTestMixin, View):    def test_func(self):        return self.request.user.is_staff
```

staff user access

6. Object / Query Related Mixins

reusable mixin Django- generic views-

6.1 MultipleObjectMixin

object handle behavior

ListView- views-

javascript.snippet

JAVASCRIPT

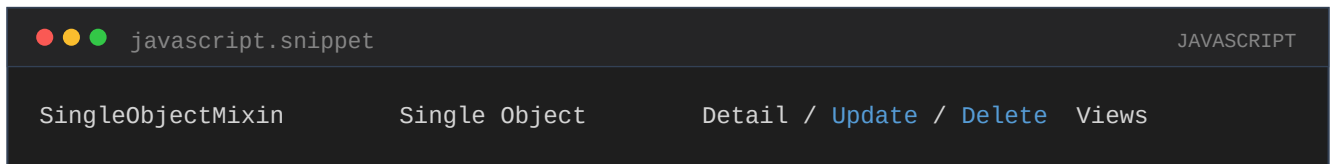
```
MultipleObjectMixin    Multiple Objects    ListView
```

pagination object list-

6.2 SingleObjectMixin

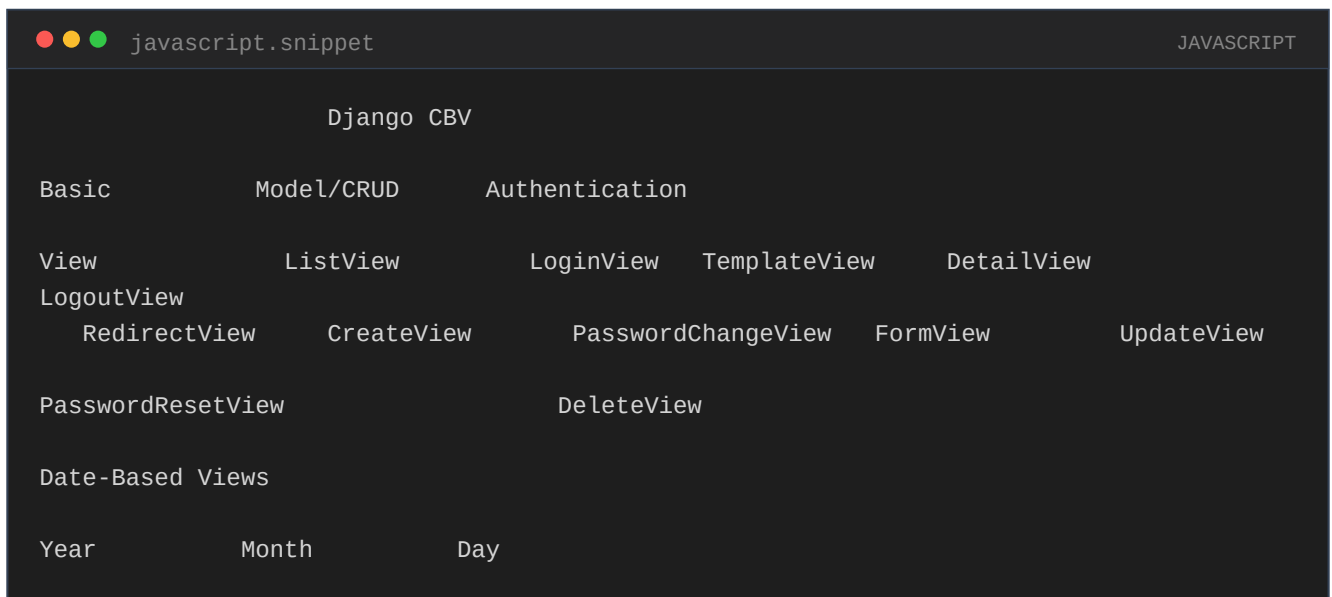
object handle reusable behavior

single-object-based generic view-



7. CBV Structure --

Django CBV- :



8. CBV + Mixins

project- CBV Mixins common

:



:

9.

View	
View	Basic/custom request handling
TemplateView	Template render
RedirectView	Redirect
FormView	Form handling
ListView	Multiple objects
DetailView	Single object
CreateView	object
UpdateView	Existing object update
DeleteView	Object delete
ArchiveIndexView	Date-based archive index
YearArchiveView	Year-based archive
MonthArchiveView	Month-based archive
WeekArchiveView	Week-based archive
DayArchiveView	Day-based archive
TodayArchiveView	Today's archive
LoginView	Login
LogoutView	Logout
PasswordChangeView	Password change
PasswordResetView	Password reset

10. Mixins

Mixin	
LoginRequiredMixin	Login required
PermissionRequiredMixin	Permission check
UserPassesTestMixin	Custom condition check
MultipleObjectMixin	Multiple objects handle
SingleObjectMixin	Single object handle

11. Interview / Exam- Shortcut

Basic Views

```

javascript.snippet
JAVASCRIPT

ViewTemplateViewRedirectViewFormView

```

CRUD Views

```

javascript.snippet
JAVASCRIPT

ListViewDetailViewCreateViewUpdateViewDeleteView

```

Date-Based Views

```

javascript.snippet
JAVASCRIPT

ArchiveIndexViewYearArchiveViewMonthArchiveViewWeekArchiveViewDayArchiveViewTodayArchiveView

```

Authentication Views

```

javascript.snippet
JAVASCRIPT

LoginViewLogoutViewPasswordChangeViewPasswordChangeDoneViewPasswordResetViewPasswordResetDoneViewPa

```

Important Mixins

```

javascript.snippet
JAVASCRIPT

LoginRequiredMixinPermissionRequiredMixinUserPassesTestMixinMultipleObjectMixinSingleObjectMixin

```

12.

CBV formula:

```
javascript.snippet JAVASCRIPT

View Basic Request HandlingTemplateView TemplateFormView FormListView Many
ObjectsDetailView
One ObjectCreateView CreateUpdateView UpdateDeleteView DeleteMixins Reusable Behavior
```

CRUD shortcut:

```
javascript.snippet JAVASCRIPT

C CreateViewR ListView / DetailViewU UpdateViewD DeleteView
```

Django CBV- core structure